Code generation from scratch

HW2 experiment report - work in progress
Andrei Vorsin | 15 October 2026

Current result

The last scored public submission achieved **18.85% (181/960)**. The public sheet subsequently showed rank 13 of 15, with first place at 78.33%. The 33% baseline has not been passed. No private leaderboard result is available. The goal of first place remains open.

Task and rules

Train an encoder-decoder Transformer from random weights using only the supplied SlavaYus/tinysms_qa_12m training split. Generate one Python program per test question. All 1,919 programs must be generated within 180 seconds on one NVIDIA T4. No pretrained weights, outside examples, model distillation, or agent-written target programs are used.

Data and architecture

The allowed dataset contains 11,838,003 question/code pairs in 60 training files. The test file contains questions only. A byte-level BPE tokenizer with 8,192 tokens was fitted on shard 0 after reserving its first 1,024 rows. The saved model has 9,967,360 parameters: four encoder layers, four decoder layers, width 256, feed-forward width 768, four attention heads, gated GELU and dropout 0.1. It uses the T5 architecture with randomly initialized weights.

Training uses AdamW, learning rate 0.0003, 500 warmup updates, cosine decay to 10% of the initial rate, weight decay 0.01 and gradient clipping at 1.0. Input and target limits are 128 and 192 tokens. Padding tokens do not contribute to the loss. Model state and optimizer state are saved at regular intervals.

Validation correction

Earlier runs excluded the first 1,024 row positions but did not exclude copies of their questions. An audit found 24 later rows in shard 0 with the same question text. Historical validation was therefore not fully independent. Future streams exclude every reserved question text across all shards. Rows 0-255 select checkpoints; rows 256-511 provide separate diagnostics. The earlier tokenizer may have seen duplicate question text.

Quality experiments

Historical runs

Experiment	Evidence	Interpretation
Local MPS / short T4 checks	One local update; ten T4 updates; later T4 pilot reached 4,000 updates, loss 1.6469.	Pipeline checks. The longer pilot checkpoint was not recovered.
Sequential shards, 20,000 updates	Final held-out token loss 1.1213; best logged 0.9207 at step 12,500.	Best weights were not retained; last weights only.
Rotate after 1,000 mini-batches, 20,000 updates	Final loss 0.9815; saved best 0.8636 at step 18,500.	Selected historical checkpoint. Held-out exact code match 2/256; public score 18.85%.

The historical comparison supports broader data coverage by token loss, but the validation-copy issue limits that conclusion. Token loss scores the next token with the reference prefix supplied; it does not show whether a generated program computes the right answer.

Audit before the new pilots

All 1,024 held-out programs round-trip through the tokenizer exactly with cleanup disabled. Only 2 held-out questions and 3 of 1,919 test questions exceed the input limit; 5 held-out programs exceed the target limit. Truncation and tokenizer corruption do not explain most failures in this audit. Samples of 1,000 rows from shards 0, 15, 30, 45 and 55 have mean code lengths 65.96, 62.56, 93.17, 61.84 and 118.95 tokens. This suggests that sequential training encounters different styles for extended periods.

Matched sampling pilot

Hypothesis: mixing original examples across shards improves held-out numeric agreement. Two fresh runs use identical initialization, model, seed, BF16 precision, batch 64, accumulation 1, grouping window 32 and 6,000 updates. Both exclude validation question texts. The control rotates after 1,000 batches. The treatment mixes up to 4,096 randomly selected original rows from every shard and resamples each epoch. Sampling is the experimental variable. Comparisons with older runs also change batch shape and validation exclusion.

Run	Best token loss	Held-out numeric agreement
control	1.6012	14/245
mixed	1.4793	16/245
mixed_lr001	1.1409	27/245

Numeric evaluation compares solve() outputs of generated and supplied held-out programs, using limited separate processes and restricted imports. Only finite numeric reference results are usable. The old saved model agrees on 50/245 usable references (20.4%). Reference errors and runner restrictions can affect this measure. Numeric outputs are never used as training targets.

Generation and speed

Measured optimization experiments

The course materials motivated length grouping, mixed precision, gradient accumulation, clipping and caching. The model already uses cached autoregressive generation. Training disables that cache. Loss accumulation stays on the GPU until logging. These choices were measured rather than assumed to make the full workload faster.

Blackwell inference, same weights	Median seconds	Changed outputs
Before, FP32	11.350	0
After, FP32	11.240	0
After, FP16	13.027	54
After, BF16	12.249	313

Two repetitions used reversed variant order on the second pass. The approximately 1% FP32 difference has overlapping timing ranges; it is not convincing evidence of a speedup. Lower inference precision was slower and changed programs. FP32 remains the default. An 80-update continuation benchmark also found no clear speed or validation-loss improvement; it does not establish full retraining quality.

T4 requirement

The historical 39.09-second T4 measurement belongs to an older run. Blackwell timings cannot establish the T4 requirement for the current candidate. The inference notebook now checks the actual GPU name and rejects a non-T4 speed verification. The final checkpoint and decoding configuration require a fresh complete T4 measurement before compliance is claimed.

Exact model output and submission

Generation sorts questions by token length and restores their original positions. Greedy decoding is followed by four-beam generation for syntactically invalid outputs; the first parseable model candidate is retained. No candidate is rewritten. Selected token IDs are saved, then decoded again to check equality with every output string. A sidecar records source commit, checkpoint hash, question-order hash, output hash, device and generation settings. JSONL conversion only wraps each exact string as a code field.

The historical submission included one handwritten identifier repair, which does not meet the user's stricter model-only condition. That repair has been removed from all future inference. The old score is retained as historical evidence, not a score for a newly verified model-only candidate.

The user confirmed that the required input is the official test.parquet question column in its original 1,919-row order. Generation preserves that order. The separate pass1_questions.jsonl is no longer a prerequisite.

Agent use and reproducibility

What the agent did

Codex read the assignment and course weeks, wrote data access, training, inference and diagnostic code, controlled Colab through MCP, inspected GPU state, compared measurements, prepared Telegram file packaging and wrote this report. It identified incomplete validation exclusion, stale T4 claims and handwritten postprocessing. Those findings changed the data exclusion rules, provenance records and speed checks.

Codex did not provide solution programs as training targets or replace model answers with its own solutions. No external model was used as a teacher. The submitted programs must come from the trained Transformer. Bot examples are not used to tune individual answers. The humanizer skill is used only to make report prose direct; it does not conceal agent involvement.

What remains open

The new pilot weights were lost when temporary runtime storage was released before export succeeded; their measurements remain saved. The revised workflow verifies Drive save/export on CPU, prepares data and source bundles there, and archives every training checkpoint and its exact predictions to Drive before bot evaluation.

The public baseline and first-place objective remain unmet. Before selecting a final model, compare held-out numeric results, preserve its checkpoint and training evidence, run full generation on one T4, and obtain an actual score from the course bot. A public first-place result would not establish private first place. No private result is claimed.

Reproduction files

Repository: OsVox/hse-nlp, branch 2026, homeworks/hw2. The directory contains train.py, data.py, numeric_eval.py, inference.py, tokenizer files, saved model weights, inference and training notebooks, experiment records and submission receipts. quality_experiments.md specifies the controlled pilot before its run. The raw logs and run configurations accompany the quality results.

Use a fresh output directory for each experiment. Run training with the recorded settings and keep latest plus best checkpoints. Run numeric evaluation on reserved training rows only. Use hw2_inference_colab.ipynb on a T4 to generate and package the test outputs. Verify every checkpoint in Drive before releasing compute; export predictions and record bot receipts from Drive.

Sources

Course assignment; allowed dataset and schema; course week 2 (encoder-decoder attention and decoding), week 3 (training methods), week 4 (attention and caching). Local measurements: metrics.json, gpu_benchmark/*.json, quality_results.json when present, and submissions/2026-10-15-gpu-check.json.